

Overleaf Studio

System Architecture & Technical Specification Document

Open-Source Real-Time LaTeX to PDF Web IDE & AI Copilot Engine

Core Engineering Team

Antigravity Advanced Agentic Coding

August 15, 2026

Abstract

This document outlines the comprehensive technical architecture, design patterns, framework allocations, and execution workflows of **Overleaf Studio**—a high-performance, open-source LaTeX to PDF Web IDE. The system unifies client-side code editing via Monaco Editor, multi-stage compilation using the standalone Tectonic LaTeX engine, automated syntax healing and box badness resolution, and an integrated AI LaTeX Copilot Agent into a portable, production-ready containerized platform.

Contents

1	System Architecture Overview	2
2	Technical Tier Breakdown	2
2.1	Tier 1: Frontend Client Presentation Layer	2
2.2	Tier 2: Backend Application & API Layer	2
2.3	Tier 3: Compilation & Auto-Healing Engine	3
2.4	Tier 4: AI Diagnostics & Copilot Agent	3
2.5	Tier 5: DevOps, Containerization & Persistent Storage	4
3	Data Flow & Execution Pipeline	4
4	Security Architecture	4
5	Conclusion	4

1 System Architecture Overview

Overleaf Studio is structured as a decoupled 5-tier architecture that guarantees sub-second compilation, fault-tolerant rendering, multi-file workspace persistence, and intelligent AI-guided code corrections.



Figure 1: High-Level 5-Tier System Architecture Diagram of Overleaf Studio.

2 Technical Tier Breakdown

2.1 Tier 1: Frontend Client Presentation Layer

- **Monaco Editor (VS Code Web Engine):** Serves as the primary LaTeX code editing surface, customized with the *Dark Emerald* Overleaf design system.
- **Intelligent Autocomplete Provider (`latex-completions.js`):** Injects over 100+ LaTeX mathematical commands, environments (`align`, `bmatrix`, `figure`), Greek letters, and dynamic scanners for `\ref{...}` labels and `\cite{...}` BibTeX entries.
- **Split-Screen PDF Controller (`pdf-viewer.js`):** Embedded zero-latency PDF rendering viewport with hardware-accelerated zoom controls and auto-compile synchronizer.
- **AI Copilot Slide-Over Drawer (`ai-copilot.js`):** Glassmorphic copilot interface with quick-action chips, prompt input, markdown explanation renderer, and 1-click *Apply to Editor & Compile* action.
- **Vanilla CSS3 Design System:** Zero-dependency, performant responsive layouts with CSS grid, flexbox, and CSS variables.

2.2 Tier 2: Backend Application & API Layer

- **FastAPI (Python 3.12):** High-throughput asynchronous REST API framework handling project CRUD, multi-file I/O, compilation requests, AI streaming, and asset uploads.

- **Uvicorn ASGI Server:** Production-grade async server managing concurrent connections and non-blocking sub-process workers.
- **Security & Authentication (`auth.py`):** Salted password hashing via `bcrypt` (12 rounds) and stateless JWT sessions with HS256 signatures.
- **SQLite Database (`database.py`):** Lightweight embedded relational database enforcing foreign key cascades across `users`, `projects`, and `project_files`.
- **Server-Side Rendering (`Jinja2`):** Fast initial page loads for auth screens, dashboards, and editor workspaces.

2.3 Tier 3: Compilation & Auto-Healing Engine

- **Tectonic LaTeX Engine (XeTeX):** Standalone, multi-package LaTeX compiler capable of on-the-fly package resolution without requiring a 4GB TeXLive install.
- **Intelligent Healer (LaTeXHealer):** Fault-tolerant multi-stage compiler pipeline that intercepts compilation failures and applies regex-based AST repairs:
 1. Literal comment operator escaping: `\text{Accuracy (%)}` $\rightarrow$ `\text{Accuracy (\%)}`.
 2. Math paragraph break cleaner: Converts illegal blank lines in math mode to comments (%).
 3. Unclosed command balancing: Auto-closes unclosed `\text{...}`, `\textbf{...}`, and bracket pairs.
 4. Unclosed environment resolution: Injects missing `\end{itemize}`, `\end{align}`, `\end{document}`.
 5. Dynamic package discovery: Auto-imports `amsmath`, `amssymb`, `graphicx`, `booktabs`, `hyperref`, `xcolor`.
- **Automated `\hbox` & `\vbox` Solver:** Automatically injects `\raggedbottom` (solving Underfull `\vbox` (badness 10000)), `\usepackage{microtype}` (solving Overfull `\hbox`), and `\emergencystretch=3em`.

2.4 Tier 4: AI Diagnostics & Copilot Agent

- **Multi-Model Orchestration (`ai_agent.py`):**
 - **Google Gemini 2.0 Flash:** Ultra-fast 1M token context reasoning via REST API and Google GenAI SDK.
 - **Free Public LLMs:** Zero-config free online model fallback (DeepSeek-V3 / Llama-3.3).
 - **Offline Heuristics Specialist:** Deterministic rule-based LaTeX expert engine running locally without internet access or API keys.
- **Structured JSON Output:** Produces standardized explanations, change summaries, and full drop-in replacement source code.

2.5 Tier 5: DevOps, Containerization & Persistent Storage

- **Docker Multi-Stage Container:** Minimal Debian-based image bundling native Linux Tectonic, font libraries, and Python 3.12.
- **Docker Compose:** Single-command orchestration mapping persistent volumes (`./data:/app/data`) and port `8000:8000`.
- **Workspace Sandboxing:** Strictly isolated user directory paths (`data/workspaces/{user_id}/{project_id}`) preventing cross-tenant access.

3 Data Flow & Execution Pipeline

1. **Editing Phase:** The user types LaTeX source in Monaco Editor. The client-side auto-complete engine suggests commands, labels, and citations.
2. **Debounced Sync:** After 1200ms of idle typing (or on `Ctrl+Enter`), the active file content is saved via `POST /api/projects/{id}/files`.
3. **Compilation Trigger:** `POST /api/projects/{id}/compile` invokes the multi-stage compiler pipeline.
4. **Stage 1 (Direct):** Tectonic compiles the document. If return code is 0, the output PDF is served immediately.
5. **Stage 2 (Auto-Heal):** If errors occur, LaTeXHealer repairs syntax bugs in memory, writes `_build_main.tex`, recompiles, and copies the resulting PDF to `main.pdf`.
6. **Live Rendering:** The PDF viewer displays the fresh PDF; compiler notices and auto-fixes are populated in the diagnostics drawer.
7. **AI Consultation:** The user can open the AI Copilot to ask questions, fix box warnings, or auto-apply refined code directly into Monaco.

4 Security Architecture

Security Domain	Threat Vector	Mitigation Strategy
Authentication	Brute-force / Credential theft	Salted <code>bcrypt</code> (12 rounds) + <code>HttpOnly</code> JWT cookies
Database	SQL Injection (SQLi)	100% Parameterized queries via SQLite driver
File System	Directory Path Traversal	<code>os.path.basename</code> sanitization on all file operations
Multi-Tenancy	Cross-User Data Access	User-ID verification middleware on all endpoints
Execution Sandbox	Command Injection	Safe sub-process argument lists (no <code>shell=True</code>)

Table 1: Security Controls & Threat Mitigation Matrix.

5 Conclusion

Overleaf Studio delivers a robust, zero-dependency alternative to complex cloud LaTeX platforms. By combining asynchronous Python backend architecture, local standalone compiler binaries, automated fault-tolerant healing heuristics, and cutting-edge AI assistance, it establishes a reliable foundation for scientific and technical document authoring.